

Soutenance de thèse

Conception d'un système mémoire pour traitement de données creuses

Thèse soutenue par

Valentin ISAAC--CHASSANDE

le 11 mai 2026

Université Grenoble Alpes, CEA List, TiMA

Plan

1. Prérequis et Problématique

2. Approche 1 : Cache de réduction

3. Approche 2 : Partitionnement du cache

4. SpDCache : Fonctionnement et Architecture Matérielle

5. Evaluations et Performances de SpDCache

Title

Subtitle

aaa
aaa
aaa
aaa
aaa
aaa aaa aaa aaa aaa aaa aaa aaa aaa aaa aaa aaa aaa aaa aaa aaa aaa aaa aaa

Prérequis

- Début de ASAP? Jusqu'à SpDC?
- OP SpMV seulement?

Cache de reduction

Utiliser la théorie pour montrer pourquoi on veut exploiter les structures des matrices

- Matrices creuses, éléments isolés -> cache devient fully associatif! Mais pas possible, et structure exploitable, de la matrice mais aussi kernel
- montrer dense spmv dans cache (introduire notion de thrashing) VS bande horizontale dans cache
- A quoi ressemblent les matrices?
- Les matrices bandées
- On prend une bande, on la met dans un cache, si on choisit correctement la taille c'est ok -> DRC

SpDCache, fonctionnement + détails RTL

Evaluations (tout, python + RTL + théorie) Attention comp entre les 3 bancaire

Mettre l'accent sur l'approche plutôt que les résultats?

Prérequis
○○○○

Réductions
○○○○○○

Irrégularités
○○○○○○

SpDCache
○○○

Evaluations
○○○○

tmp
○○

Références

End
○

aaa

aaa

Plan

1. Prérequis et Problématique

2. Approche 1 : Cache de réduction

3. Approche 2 : Partitionnement du cache

4. SpDCache : Fonctionnement et Architecture Matérielle

5. Evaluations et Performances de SpDCache

SpMV

OP

reductions

irrégulières

scatter

formats creux CSC

matrices creuses

correspondance direct vs associatif : 1 schéma pour expliquer les 3 types
visu du problème dans cache classique (au niveau, voir ASAP)

Cache : mémoire intermédiaire entre le processeur et la mémoire principale

Fonction : stocker quelques données proche du processeur

Pourquoi : une transaction en mémoire principale prend du temps (~100 ns)

Problème :

Comment traiter efficacement des réductions irrégulières ?

Deux approches complémentaires :

Approche 1 : Cache de réduction pour calculer les réductions dans le cache et éviter des transactions inutiles

Approche 2 : Traiter l'irrégularité en s'appuyant sur la structure des matrices, partitionnement du cache

Plan

1. Prérequis et Problématique

2. Approche 1 : Cache de réduction

- Cache générique
- Cache de réduction
- Conclusion

3. Approche 2 : Partitionnement du cache

4. SpDCache : Fonctionnement et Architecture Matérielle

5. Evaluations et Performances de SpDCache

Cache générique, fonctionnement (rapide)

Réduction dans cache générique

Cache de réduction, qu'est ce qu'il y a en plus

Réduction dans cache de réduction VS générique

Diminution du nombre de transactions mémoires / Non bloquant (RED envoyée et oubliée) /

Transactions mémoires du haut vers le bas

Cache générique

Fonctionnement & Réduction dans un cache générique

Cache classique : set-associatif, plusieurs voies

Granularité : ligne de cache (64 o)

Transactions mémoires : lecture et écriture de ligne de 64 o

Réduction : lecture dans le cache (avec hit, miss ou évincement), accumulation dans le processeur, écriture dans le cache



<schéma d'une réduction dans un cache générique>

Cache de réduction

Qu'est-ce ?

Même caractéristiques que cache : set-associatif, plusieurs voies, lignes de cache

Traite des instructions de réduction RED

Accumulation sur place, ajout d'un accumulateur

Style write-back

Cache de réduction

Une réduction dans un cache de réduction



list

<schéma d'une réduction dans un cache générique>



list

<schéma d'une réduction dans un cache de réduction>

Comparaison du nombre d'instruction / le processeur n'attend pas les REDs

Conclusion

Notre cache doit supporter les réductions

Note : existant, PHI, SortCache, ASA le font déjà

Plan

1. Prérequis et Problématique

2. Approche 1 : Cache de réduction

3. Approche 2 : Partitionnement du cache

4. SpDCCache : Fonctionnement et Architecture Matérielle

5. Evaluations et Performances de SpDCCache

aaa

Figure, thrashing

Pour éviter, et cache fonctionne bien, petite matrice, ou bande horizontale (montrer sur la figure)

Pas en pratique

Mais cas proche existe, bandée (montrer sur la figure)

Si juste bandée, pas ok, change rien

Si parfaitement bandée -> cache ok, moyennant une formule

Qu'est ce que ca donne apres étude sur le mem traffic -> 2 ordres de grandeur

Idee : isoler une bande diagonale de la matrice -> DRC

Multiplication Matrice-Vecteur dans un cache générique :

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ❌



list

Multiplication Matrice-Vecteur dans un cache générique :

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ✖

Petite matrice



Multiplication Matrice-Vecteur dans un cache générique :

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ❌

Petite matrice, bande horizontale :



Multiplication Matrice-Vecteur dans un cache générique :

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ❌

Petite matrice, bande horizontale :

- Tiens dans le cache, pas d'évincement
- Le cache est utile ✔
- Cas peu intéressants ❌



Multiplication Matrice-Vecteur dans un cache générique :

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ❌

Petite matrice, bande horizontale :

- Tiens dans le cache, pas d'évincement
- Le cache est utile ✔
- Cas peu intéressants ❌

Matrice bandée :

- Évincements répétés dans le cache



Multiplication Matrice-Vecteur dans un cache générique :

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ❌

Petite matrice, bande horizontale :

- Tiens dans le cache, pas d'évincement
- Le cache est utile ✔
- Cas peu intéressants ❌

Matrice bandée :

- Évincements répétés dans le cache
- Le cache n'est pas utile ❌
- Structure très courante ✔



Multiplication Matrice-Vecteur dans un cache générique :

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ✖

Petite matrice, bande horizontale :

- Tiens dans le cache, pas d'évincement
- Le cache est utile ✔
- Cas peu intéressants ✖

Matrice bandée :

- Évincements répétés dans le cache
- Le cache n'est pas utile ✖
- Structure très courante ✔

Matrice parfaitement bandée :

- Nombre d'évincements limité
- Le cache est utile ✔
- Structure peu courante ✖



Traffic mémoire théorique avec des matrices bandée

c

Traffic mémoire du vecteur de sortie
versus

densité en dehors de la bande,
pour différentes tailles de cache

- Densité dans la bande fixe (10^{-2})
-
- ➔ Réduction de deux ordres de grandeur
 - SI Matrice parfaitement bandée
 - ET Largeur de bande suffisamment petite par rapport à la taille du cache

<relation>



(graph avec courbes qui apparaissent au bon moment)

CONCLUSION

On sélectionne une bande diagonale de la matrice, qu'on traite avec un cache set-associatif

- <relation>

Que faire du reste, hors de la bande ?

Granularité : Éléments isolés

- Incompatible avec les lignes de cache
- ➔ Pas de ligne de cache :
64o (8 mots) → 8o (1 mot)

Absence de structure claire

- Nombre de *hits* bas
- ➔ Pour augmenter la quantité de *hits* dans un espace de cache modeste :
Cache complètement associatif

Transactions mémoires inadaptées

- Lectures et écritures sur 64o (8 mots)
- ➔ Transactions mémoires "par éléments"
RMW atomique sur 8o (1 mot)



(graph avec courbes qui apparaissent au bon moment)

CONCLUSION

On traite les éléments hors-bande de la matrice avec un cache « par élément », complètement associatif, avec des transactions mémoires « par élément » (RMW)

Conclusion

Deux régions pour traiter le dense et le creux

C'est ma contribution

Plan

1. Prérequis et Problématique

2. Approche 1 : Cache de réduction

3. Approche 2 : Partitionnement du cache

4. SpDCCache : Fonctionnement et Architecture Matérielle

5. Evaluations et Performances de SpDCCache

Fonctionnement de SpDCache et Techniques HW

Faire le lien : deux régions annoncées, plus région générique

Macro Architecture

Expliquer comment les deux régions sont gérées

Quelques points d'implémentation RTL (partition du cache au moins)

Slide sur flexibilité, config, RISC-V, opensource(?)

Prérequis
○○○○

Réductions
○○○○○○

Irrégularités
○○○○○○

SpDCache
○○●

Evaluations
○○○○

tmp
○○

Références

End
○

text

Plan

1. Prérequis et Problématique

2. Approche 1 : Cache de réduction

3. Approche 2 : Partitionnement du cache

4. SpDCache : Fonctionnement et Architecture Matérielle

5. Evaluations et Performances de SpDCache

RTL et python en meme temps

Traffic mémoire

Config 25/50/25

Puis accélération (Perf peu impressionnante à cause du setup minimaliste, mais une aubène pour ma thèse car permet d'analyser : on a des cas compute bound, ce qui n'arrive pas en pratique)

Répartition des matrices sur l'échelle dcd

Prérequis
○○○○

Réductions
○○○○○○

Irrégularités
○○○○○○

SpDCache
○○○

Evaluations
○○●○

tmp
○○

Références

End
○

text

Prérequis
○○○○

Réductions
○○○○○○

Irrégularités
○○○○○○

SpDCache
○○○

Evaluations
○○○●

tmp
○○

Références

End
○

text

- text $\LaTeX$ text
 - ▶ aaa
 - aaa
- text
- text
<https://www.latexstudio.net/archives/4051.html>
- text **link** [unk15]text
- text ppt :**text**

There Is No Largest Prime Number

The proof uses *reductio ad absurdum*.

Theorem

There is no largest prime number.

Démonstration.

- ❶ Suppose p were the largest prime number.
- ❷
- ❸
- ❹ But $q + 1$ is greater than 1, thus divisible by some prime number not in the first p numbers. □

There Is No Largest Prime Number

The proof uses *reductio ad absurdum*.

Theorem

There is no largest prime number.

Démonstration.

- ❶ Suppose p were the largest prime number.
- ❷ Let q be the product of the first p numbers.
- ❸
- ❹ But $q + 1$ is greater than 1, thus divisible by some prime number not in the first p numbers. □

There Is No Largest Prime Number

The proof uses *reductio ad absurdum*.

Theorem

There is no largest prime number.

Démonstration.

- ❶ Suppose p were the largest prime number.
- ❷ Let q be the product of the first p numbers.
- ❸ Then $q + 1$ is not divisible by any of them.
- ❹ But $q + 1$ is greater than 1, thus divisible by some prime number not in the first p numbers.



There Is No Largest Prime Number

The proof uses *reductio ad absurdum*.

Theorem

There is no largest prime number.

Démonstration.

- ❶ Suppose p were the largest prime number.
- ❷ Let q be the product of the first p numbers.
- ❸ Then $q + 1$ is not divisible by any of them.
- ❹ But $q + 1$ is greater than 1, thus divisible by some prime number not in the first p numbers. □

The proof used *reductio ad absurdum*.

[unk15] unknown. Thu beamer theme. 2015.

Thanks !